

What is artificial intelligence?

Understand the idea. Look inside the system.

Learn when it belongs in your work.

🕒 45-60 min reading · Beginner friendly · 16 sections

You probably meet AI through a chat window. But a conversation is only one interface to a much larger field. An email filter, a route planner, an image recognizer, and a report-writing assistant can solve very different problems. This chapter gives you a way to look inside those systems and ask useful questions about them.

Your destination

By the end, you should be able to explain AI in your own words, distinguish explicit rules from learned patterns, trace an input through an AI application, and choose a sensible first approach to a practical problem.

Before you begin: No AI background or advanced mathematics is needed. The optional Python example assumes only that you can follow an `if` statement. Allow roughly 45-60 minutes for reading and another 30-45 minutes for experiments and exercises; take longer whenever you need it.

1. Begin with a problem, not a technology

Imagine that you receive 150 support emails each morning. Some report a login issue, some ask a routine question, and a few describe a service outage. You want the urgent messages to reach the right person quickly.

There are several possible solutions. You could ask senders to choose a category in a form. You could search for a few phrases. You could train a model on previously classified messages. You could use a language model to suggest a category and draft a summary. You could also combine these approaches.

Notice what comes first: **a task and a desired result**. “Use AI” does not tell us what success looks like. “Reduce the time needed to route support messages while keeping missed outages below an agreed limit” is a much clearer objective.

The difference matters because a technically impressive system can still solve the wrong problem. A beautifully written summary is not useful if it changes the incident time. A fast classifier is not useful if it routinely sends outages to the routine queue.

For any example in this handbook, learn to identify four things:

Input: What information is available? For instance, the email text and its timestamp.

Task: What transformation or decision is required? For instance, assigning a category.

Output: What does the system produce? For instance, “login issue” and a score.

Success criterion: How will we decide whether it helped? For instance, correct routing on new messages and fewer missed urgent cases.

Think before reading on. If a support form can make the sender choose the correct category reliably, would you still need a model to infer it? Sometimes improving the input process removes the need for AI entirely.

2. What does “artificial intelligence” mean?

For this handbook, **artificial intelligence is the field of building computational systems that perform tasks involving abilities such as perception, language, reasoning, planning, learning, and decision-making**. This is a working definition, not a claim that there is one universally agreed boundary around the field. An agent-oriented view also asks how a system chooses actions in an environment; see [Poole and Mackworth’s introduction to AI](#).

“Artificial” means that the capability is implemented in a constructed system. “Intelligence” points to the kinds of tasks we want that system to carry out. It does not, by itself, establish that the system is conscious, has feelings, understands exactly as a person does, or is capable of every intellectual task.

A useful analogy is flight. An aircraft can fly without flapping its wings like a bird. Likewise, a program can perform a task associated with intelligence without reproducing every mechanism of a human mind. The analogy explains why we should examine the mechanism and the evidence for a capability rather than assume human-like properties from a familiar-looking result.

Capability is specific to a task

A system may recognize objects in photographs yet be unable to plan a trip. A system that produces good answers about programming may still make errors on a particular calculation. Success on one task is evidence about that task under the tested conditions. It is not automatic evidence of unrestricted competence.

In this handbook, **capability** means an ability demonstrated on a defined task. **Reliability** means how consistently the system meets requirements across relevant situations. A system can demonstrate a capability in a few examples without being reliable enough for everyday use.

AI is broader than learning from data

Some AI methods search possible solutions, apply logical rules, or plan sequences of actions. Others learn patterns from examples. **Machine learning, or ML, is a major approach within AI in which a model is fitted using data.** It is not a synonym for every AI method. Google's [introductory ML explanation](#) provides a complementary introduction to learning models from data.

We will build the full map of AI, ML, deep learning, generative AI, language models, retrieval, and agents in Chapter 2. For now, remember the distinction between **the task we want to solve** and **the method we use to solve it**.

3. Look inside an AI application

Consider a system that suggests the category “service outage” for a support message. The categorization component is only one part of the application. Something must collect the message, prepare it, run the categorization, apply routing rules, and show the result to a person.

➤ An AI application has several responsibilities

FROM INFORMATION TO A USEFUL RESULT



Figure 1. A model output becomes useful through preparation, application policies, and review. Feedback can inform later improvements; it does not automatically retrain the model.

Read the diagram from left to right. An application receives information, prepares it, obtains a result from a model or another decision mechanism, and uses ordinary software rules to determine what happens next. Feedback can be collected for later evaluation and improvement.

Here are the components in concrete terms:

Component	Meaning	Support-message example
Data	Information represented in a form software can process	Message text, time, selected service
Preparation	Converting or checking information before use	Checking required fields and removing duplicated quoted replies
Algorithm	A specified procedure for performing a computation	A search procedure or a procedure that fits a model
Model	A representation used to produce an estimate or output; in ML, its adjustable values are fitted using data	A classifier that maps a message to category scores
Application logic	Code that decides how results are used	Send uncertain cases to a review queue
Interface	Where a user supplies input or sees results	An email tool or a support dashboard
Evaluation	Checking the system against defined requirements	Compare suggested categories with reviewed answers

An **algorithm** and a **model** are related but different. A training algorithm describes how to adjust a model. The resulting model contains the fitted state used to produce outputs. A recipe is loosely like a procedure; the fitted model is a particular result of executing a learning procedure on particular data. Like all analogies, this is only a starting point, not a technical equivalence.

A model result is not the same as an action

Suppose the model gives a message an urgency score of 0.82. That is an output. Sending a notification is an action. The application might notify a person above a chosen cutoff, request review in an intermediate range, or do nothing below it.

The cutoff and the routing policy are design decisions. The model does not automatically decide the organization's priorities. Similarly, an assistant can draft a database query while application code restricts it to approved read-only operations.

Feedback does not automatically mean learning

Clicking "wrong category" may store feedback. It does not necessarily change the model immediately. A team might review those records, update the dataset, train a new version, evaluate it, and only then deploy it. Whether feedback changes a system depends on how the application was built.

4. The different kinds of work AI can do

It helps to describe capabilities using verbs. The table below is a task map, not a list of mutually exclusive technologies. A single application can combine several of them.

Capability	What it means	Everyday or workplace example	What to check
Perceive	Extract useful information from sensory input	Read text from a scanned form or identify an object in an image	Does it work on blurry, unusual, or partly hidden inputs?
Classify	Assign an input to a category	Label a support message as login, payment, or outage	Which categories are confused?
Estimate	Produce a numerical estimate	Estimate the time needed to resolve a ticket	How far are the estimates from observed values?
Rank	Put candidates in an order	Put the most relevant help articles first	Are useful results near the top?
Generate	Produce content	Draft a report from incident notes	Are the statements supported and the requested details preserved?
Reason or solve	Apply relationships or constraints to reach a result	Check which release steps satisfy dependencies	Is the answer valid, and can it be verified?
Plan	Choose a sequence of steps toward an objective	Order troubleshooting actions	Are the steps feasible and safe to execute?
Learn	Adjust a model from experience or data	Fit a classifier using reviewed support tickets	Does the improvement transfer to new examples?

Prediction does not always mean forecasting the future. A model that identifies the language of an existing sentence is making a prediction about an unknown label. A forecast is specifically an estimate about a future event or quantity.

Perception is not simply receiving data. A camera receives pixels; recognizing a damaged label in those pixels is a perception task. Likewise, accepting an audio file is input handling; transcribing its speech is a recognition task.

Planning and execution are different. A system may propose restarting a service. Actually restarting it requires an authorized tool call or human action. We will keep this boundary visible throughout the handbook.

5. Explicit rules versus learned patterns

Suppose you want to detect unwanted email. One starting point is an explicit rule: flag a message if it contains the phrase "claim your prize." A developer chooses the condition. The computer applies it exactly as programmed.

```
def should_flag(message):  
    return "claim your prize" in message.lower()
```

For "CLAIM YOUR PRIZE now", this returns `True`. For "A reward is waiting; send your bank details", it returns `False`. For "Training note: never trust 'claim your prize' emails", it returns `True`, even though the message is a warning about scams.

The rule is easy to inspect. It also fails when unwanted messages use different wording or legitimate messages quote the trigger phrase. Adding more rules can help, but maintaining a growing set of exceptions can become difficult.

The learned approach

In a learned approach, you collect examples and an appropriate learning procedure fits a model. For a supervised classifier, examples include the input message and a target label, such as `spam` or `legitimate`. The fitting procedure adjusts numerical values inside the model so its outputs better match the training examples under a chosen objective.

Those adjustable values are called **parameters**. A **feature** is an input property represented for the model, such as a word count or a numerical representation of the message. The developer still chooses or controls many things: the task, available data, model family, training procedure, and evaluation method. Learning does not remove human design choices.

➤ Two ways to specify behavior

EXPLICIT RULES



LEARNED PATTERNS



Figure 2. Explicit rules encode chosen conditions directly. Learning fits a model from examples. Both approaches still require evaluation.

The upper lane shows knowledge expressed directly as conditions. The lower lane separates fitting a model from using it on a new message. A learned model may detect patterns beyond a single phrase, but whether it actually does so well must be tested.

Question	Explicit rules	Learned model
Where does the decision procedure come from?	People specify conditions directly	A learning procedure fits parameters using data
How do you improve it?	Edit rules and check their consequences	Improve data, model, training, or surrounding logic; evaluate a new version
Typical strength	Clear behavior for a well-specified condition	Can capture patterns that are difficult to enumerate manually
Typical weakness	Brittle wording matches and many exceptions	Can learn misleading patterns and fail on unfamiliar inputs
Does it require evaluation?	Yes	Yes

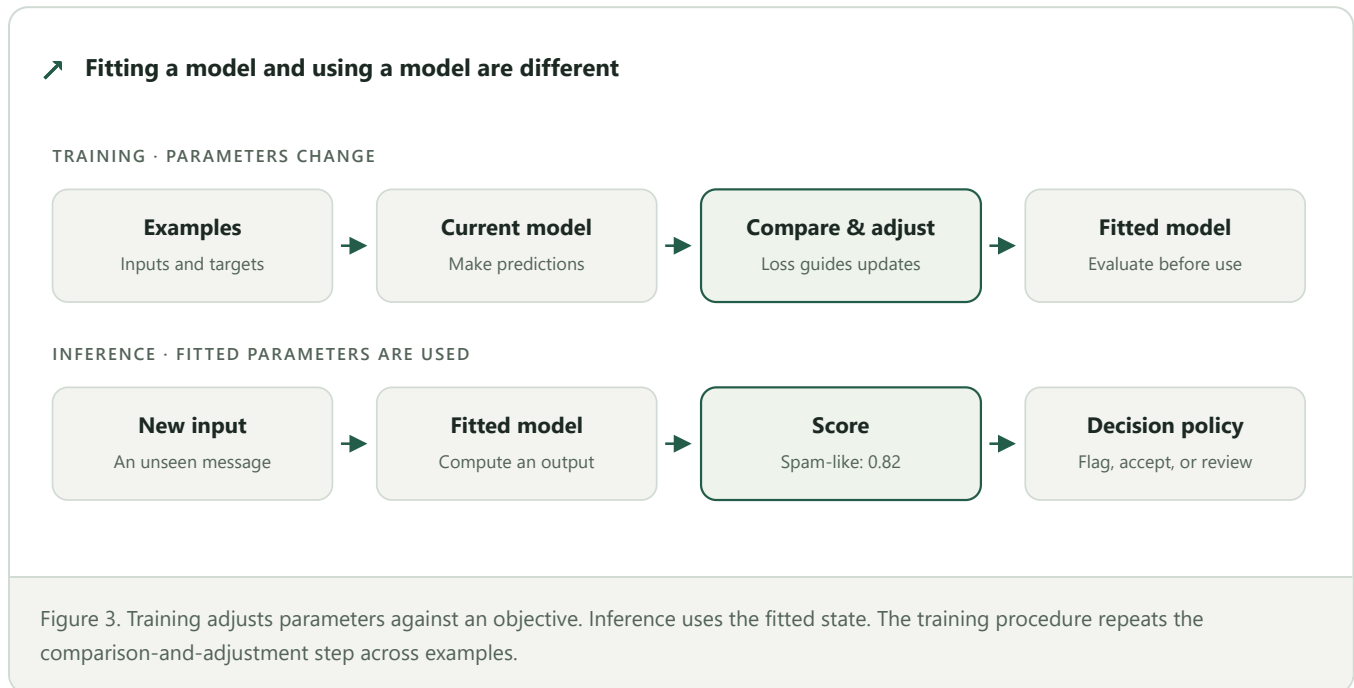
The boundary is not “rules are never AI”

Classical AI includes rule-based expert systems and search. A tax formula or a simple phrase filter is usually described as ordinary automation; a system reasoning over a knowledge base may be described as symbolic AI. There is no useful universal test based only on whether the code contains `if` statements.

For engineering, ask the more precise question: **Which part of this system is explicitly specified, which part is learned, and how is each part checked?** Most useful AI applications contain plenty of ordinary software.

6. Training and inference: two different moments

Training is the process of fitting a model using data. **Inference** is using a fitted model to produce an output for an input. The term “inference” here refers to running the model; it does not necessarily mean formal logical deduction.



In the training lane, the system makes outputs on examples and compares them with the training objective. A **loss** is a numerical measure used to express how poorly the outputs meet that objective. A training procedure uses this information to adjust parameters. We will study the mathematics of those adjustments later.

In the inference lane, a new message enters the fitted model and receives a score or label. For many deployed systems, the parameters stay fixed during this step. The application can still vary its input, retrieve new information, or apply different rules around the model.

Why new examples matter

Imagine a student who memorizes the answers to ten practice questions. They score perfectly when asked the same ten questions again. That result does not show that they can solve a new question using the underlying idea.

Likewise, **generalization** means performing usefully on new cases drawn from the situations we care about. Testing only on examples used to fit a model can give an overly optimistic impression. We set aside examples for evaluation and consider whether those examples resemble real usage.

If a spam model learned mostly from messages in one language, performance in another language is an open question. If it learned that a particular sender is always unwanted, it might rely on that shortcut instead of the message meaning. Such behavior is a reason to inspect failures, not simply increase the amount of data without examining it.

Overfitting is fitting the training data so closely that performance on relevant new data suffers. **Distribution shift** means the characteristics of incoming data differ from those represented during development. These are previews of concepts we will examine carefully in the ML chapters.

One useful distinction. Supplying a document to an assistant can change the information available for that response without changing the model's parameters. New context and new training are different mechanisms.

7. Worked example: a score becomes a decision

Now let us make the model-to-action boundary visible. Suppose a hypothetical spam classifier has produced scores for eight messages. Higher scores mean the model rates the message as more spam-like. The labels were assigned for this teaching example.

These messages, labels, and scores are synthetic. No model was trained to produce them. They are fixed illustrative values so you can reason about the decision rule without needing to learn a training algorithm first. The scores are not claimed to be calibrated probabilities: 0.80 does not automatically mean an 80% real-world chance of spam.

ID	Message excerpt	Reviewed label	Illustrative score
A	"Claim a cash prize; send your details."	Spam	0.96
B	"Urgent reward: pay a release fee."	Spam	0.88
C	"Your subscribed weekly deals newsletter."	Legitimate	0.73
D	"Account reward: verify on this unknown link."	Spam	0.67
E	"Free workshop from your registered training team."	Legitimate	0.54
F	"A private opportunity; reply with account details."	Spam	0.45
G	"Tomorrow's project meeting agenda."	Legitimate	0.21
H	"Here is the report you requested."	Legitimate	0.08

The decision rule is: **flag the message if its score is greater than or equal to a threshold**. A threshold is simply the cutoff we select. Equality counts as flagged in this example; stating that boundary prevents ambiguity.

At a threshold of **0.70**, messages A, B, and C are flagged. A and B are genuinely spam, so those decisions are correct. C is legitimate, so flagging it is a **false positive**. D and F are spam but are not flagged, so they are **false negatives**. E, G, and H are correctly left unflagged.

That gives us two correctly detected spam messages, one false alarm, two missed spam messages, and three correctly accepted legitimate messages. Five of the eight decisions are correct:

$$\text{Accuracy} = \frac{\text{correct decisions}}{\text{all decisions}} = \frac{5}{8} = 62.5\%$$

The fraction means “five correct decisions divided by eight total decisions.” Multiplying 0.625 by 100 expresses it as a percentage. Accuracy is useful here as a counting exercise, but it hides which mistakes occurred. The different meanings of accuracy, precision, and recall are explained in [Google’s classification metrics lesson](#); we will calculate those measures in depth in Chapter 15.

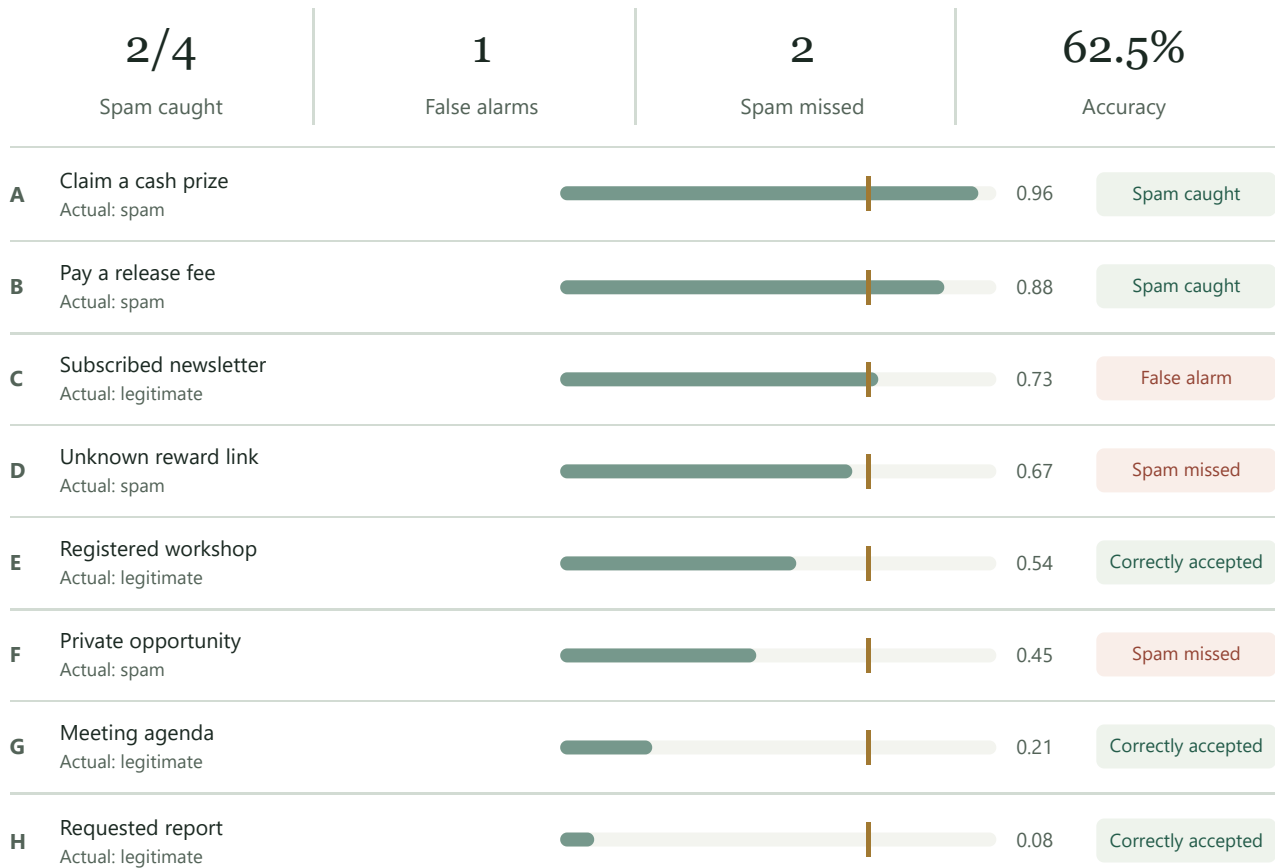
Experiment: move the cutoff

Interactive experiment 01

Synthetic data · no training

Same scores. Different decisions.

Move the cutoff. Notice which mistakes appear and disappear.



The line marks your threshold. The bars are fixed scores, not guaranteed probabilities. Flagging is a policy choice; moving this slider does not train a model.

Print snapshot: at 0.70, A, B, C are flagged; 2 spam caught, 1 false alarm, 2 spam missed. At 0.40: 4 caught, 2 false alarms, 0 missed.

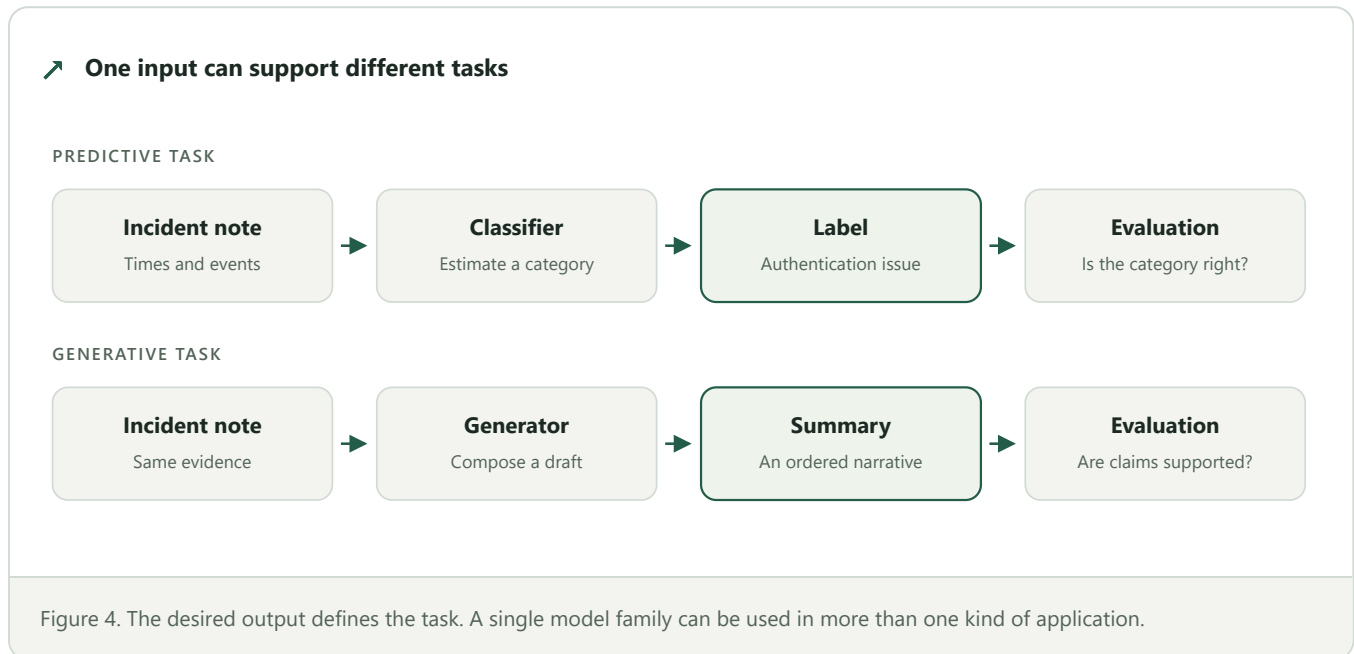
Lower the threshold to **0.40**. A through F are now flagged. All four spam messages are caught, but two legitimate messages are also flagged. Six decisions are correct, giving 75% accuracy on these eight examples. Raising the threshold to **0.90** flags only A: fewer legitimate messages are interrupted, but more spam is missed.

The scores have not changed. The parameters have not changed. **Only the decision policy has changed.** This is why a model and the policy around it must be evaluated together.

There is no universally best threshold. In one application a false alarm might be an inconvenience; in another it might interrupt an essential workflow. A missed case also has a cost. Choosing a threshold requires a representative validation set and a clear account of these trade-offs. Do not select a production policy from this tiny teaching dataset.

8. Predictive tasks and generative tasks

A **predictive task** asks for an estimate, category, score, or ranking. A **generative task** asks for content, such as a paragraph, an image, audio, or code. These descriptions concern the output required by the application. They do not imply that the underlying mathematics has no overlap.



Consider one fictional incident note:

At 10:02, users reported login failures. At 10:07, the team restarted the authentication service. By 10:12, login checks passed. The underlying cause is still under investigation.

A classification task might output **“authentication incident.”** A numerical prediction might estimate a resolution duration, using a suitable model and relevant evidence. A generation task might produce a three-sentence incident summary from the note.

A grounded summary could say: “Users reported login failures at 10:02. The authentication service was restarted at 10:07, and login checks passed by 10:12. The underlying cause remains under investigation.” It preserves the supplied facts and makes the unknown explicit.

A poor summary could say: “A database connection leak caused a ten-minute outage, which a restart permanently resolved.” The note did not establish a connection leak, the exact outage start, or a permanent resolution. The second summary invents conclusions even though it sounds plausible.

Generation does not mean factual verification

A generated sentence can be grammatically good and unsupported. **Grounding** means tying an answer to appropriate supporting information. A **hallucination**, in this context, is generated content that is incorrect or unsupported while being presented as an answer.

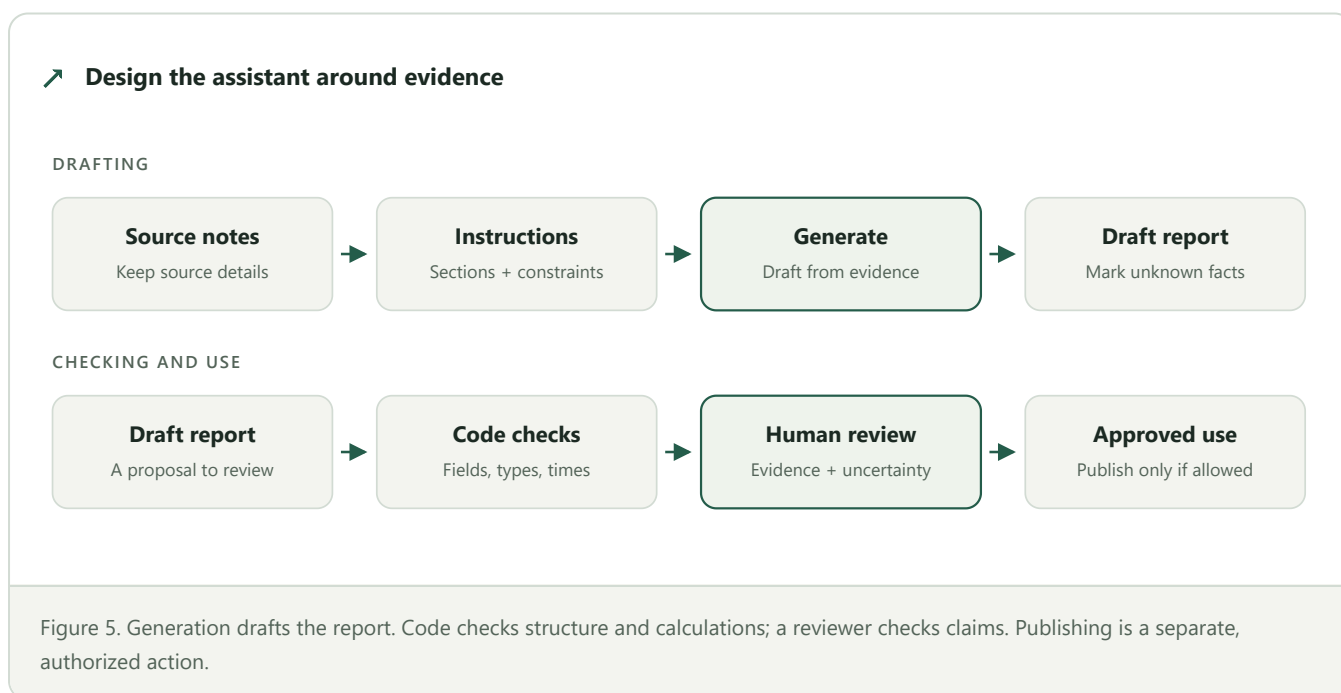
A model can also omit a crucial fact, merge events incorrectly, or express uncertainty too confidently. Evaluation must match the task: a summary needs checks for support, coverage, and preservation of meaning, not just correct spelling.

The categories can overlap

A language model often generates text by repeatedly predicting a next token, where a **token** is a unit of text represented by the model. At the application level, the result is generated content. The same model may also be prompted to output one label, making it part of a classification application. We will unpack tokens and generation later; for now, do not confuse a model's internal operation with the task its application performs.

9. Real-world walkthrough: an incident-report assistant

Let us turn the previous example into a small system design. The user wants a consistent draft containing an overview, a timeline, confirmed actions, and unresolved questions. The system should help organize evidence while keeping factual responsibility visible.



Step 1: define the required result

The report must preserve times, distinguish observations from explanations, and mark missing facts. A successful draft is one that a reviewer can check against its sources. "Sounds professional" is a useful presentation preference but is not an adequate factual success criterion.

Step 2: supply the available evidence

The application receives the incident note and, if available and authorized, related records. It should retain enough source information for a reviewer to trace important claims. Search or retrieval can provide relevant records; using retrieval to support generation is the idea behind RAG, which we will study later.

Step 3: request a structured draft

For example, an instruction could ask for an overview, a chronological timeline, and an explicit "unknowns" section. It should tell the model to use only supplied evidence for incident facts and to state when the cause has not been established.

These instructions improve the specification, but they are not proof that the result is correct. The application still needs validation and review.

Step 4: validate what code can check

Ordinary software can check that required fields exist, timestamps have an expected format, and the output can be parsed. It can compute **10:12 minus 10:02 = 10 minutes** as the interval between the first recorded report and the later successful check.

That interval is not automatically the exact duration of impact. The service might have failed before the first report or recovered before the successful check. A calculation can be arithmetically correct while its interpretation is wrong. The application should label it as the interval between recorded observations.

Step 5: review claims and uncertainty

A reviewer checks whether the draft invents a cause, implies that a restart proves causation, omits an important action, or silently turns "under investigation" into "resolved permanently." The reviewer can then correct or approve the draft.

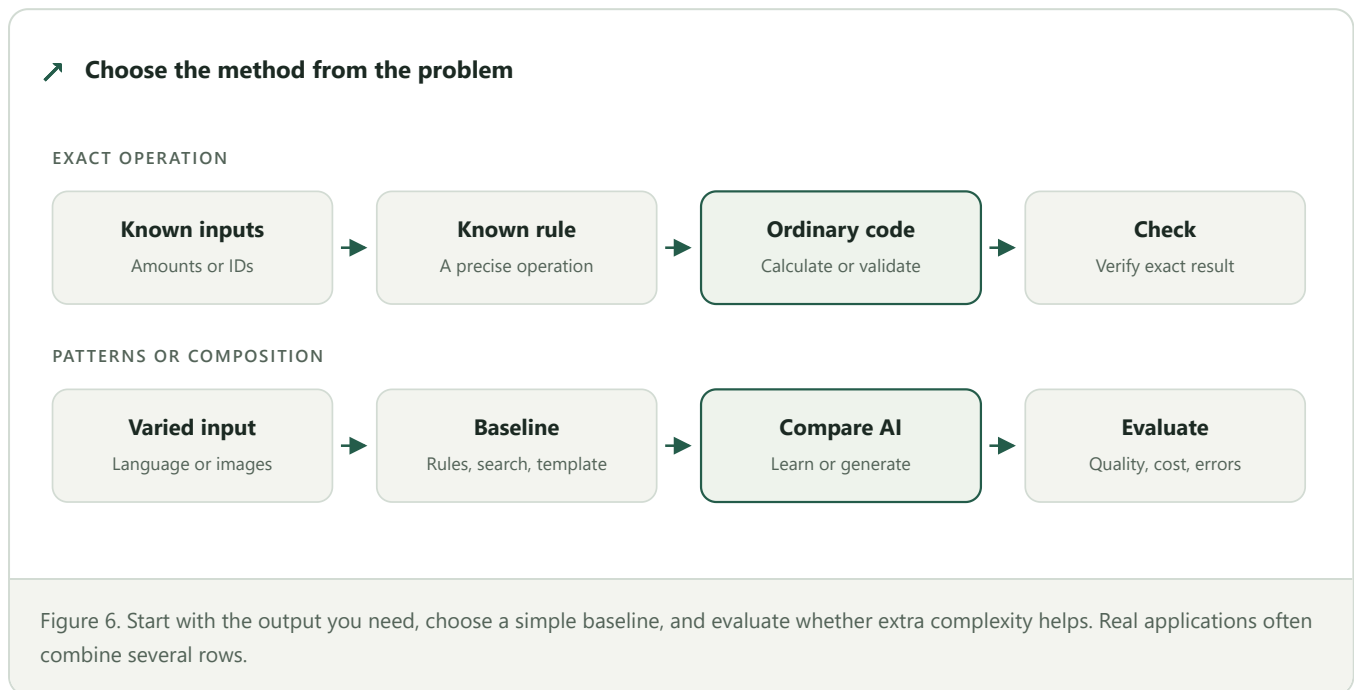
Step 6: learn from errors deliberately

Keep an appropriately handled set of examples and reviewed outputs. Identify repeated failures: invented causes, missing events, changed times, or inconsistent sections. Improve the prompt, preprocessing, validation, or model choice, then compare the revised system on examples not used to tailor the change.

This system combines language generation, search when needed, deterministic calculations, validation, and human judgment. Calling the whole application "AI" should not hide those separate responsibilities.

10. When should you use AI?

Begin with a simple baseline: a reference solution against which a more complex approach is compared. A baseline might be manual work, a keyword search, a fixed template, or a small rule-based program. It tells you whether the additional complexity produces a useful improvement.



This diagram is a starting guide, not a universal rule. Walk through the examples below and pay attention to the reason for each choice.

Problem	Sensible first approach	Why
Compute the sum of known invoice line items	Ordinary code or a spreadsheet formula	The operation is precisely defined and should be exact
Reject an empty mandatory field	A validation rule	There is no need to infer a known condition
Find a known error code in a log	Exact search	The target is explicitly specified
Find documentation for a question expressed in varied language	Search; evaluate semantic retrieval if needed	Relevant wording may differ from the question
Categorize varied free-text support requests	Compare rules, a learned classifier, and a language model	Language variation can make manual rules difficult to maintain
Draft a narrative from rough notes	A template or generative model, with evidence checks	The desired output includes language composition

Problem	Sensible first approach	Why
Permanently change a production record	Authorized application logic and an explicit policy	Generating a suggestion does not grant permission to execute it

Ask five design questions

Is the task well-defined? If people disagree about what the output should mean, resolve that ambiguity before selecting a model. Otherwise, evaluation becomes disagreement hidden behind a score.

Is the necessary information available? A model cannot reliably recover a missing incident cause merely because it writes confidently. Sometimes collecting a missing measurement is the right next step.

What is the cost of an error? A mistaken draft title and an incorrect production change need different controls. The appropriate amount of review depends on the actual consequence.

Can we measure usefulness? Define what counts as a correct result, an acceptable delay, and a tolerable review burden. An application that saves drafting time but doubles verification work may not help overall.

Can a simpler approach satisfy the need? Simpler systems are often easier to inspect and maintain. Choose complexity for a demonstrated benefit, then retain a baseline for comparison as the system changes.

11. Limitations and misconceptions

“If it sounds certain, it must be correct.”

Tone is part of the output, not independent evidence. Check factual claims against appropriate sources or deterministic computations. A statement can be both clear and wrong.

“More data always fixes the problem.”

More examples of the wrong cases can reinforce the wrong pattern. Missing groups, incorrect labels, duplicates, or information unavailable at real prediction time can undermine a dataset. The relevance and quality of the data matter alongside its quantity.

“A learned model must give a different answer every time.”

Many learned models produce the same result for the same input and model state under deterministic execution. Some generation settings introduce sampling, and execution details can also affect reproducibility. Learned versus hand-coded is a different distinction from deterministic versus variable output.

“It worked on my example, so it is ready.”

One success demonstrates one success. Try typical cases, boundary cases, incomplete inputs, unusual phrasing, and examples the system should decline to answer. Keep some evaluation examples separate from the examples used during development.

“A model replaces the whole application.”

Input handling, permissions, source retrieval, calculations, output validation, interfaces, monitoring, and recovery remain necessary. A model is one component in a wider system.

“If two events happen together, one caused the other.”

Correlation is an observed association. Causation is a claim that changing one factor affects another under appropriate conditions. In our incident example, recovery after a restart is an observation. It does not, by itself, identify the fault or establish that the restart was the only reason for recovery.

“A good score is enough.”

A score summarizes a particular test. It can hide different error types, exclude important situations, or use examples unlike actual usage. Ask what was measured, on which inputs, with which labels, and what the remaining failures mean for users.

For your learning projects, use public or synthetic examples initially. When applying the ideas at work, use an environment and data access appropriate to that work. These are practical design considerations, not a reason to avoid experimentation.

12. Optional Python lab: inspect a simple baseline

This small program shows the whole behavior of a phrase rule. You do not need to run it to understand the chapter. If you do run it, Python 3 and its standard library are sufficient; no account, model, or external package is needed.

```
def should_flag(message):
    """A deliberately limited, hand-written phrase rule."""
    return "claim your prize" in message.lower()

examples = [
    ("CLAIM YOUR PRIZE now", True),
    ("A reward awaits; send your account details", True),
    ("Training: avoid messages saying 'claim your prize'", False),
    ("Tomorrow's project meeting agenda", False),
]

correct = 0
for message, expected in examples:
    predicted = should_flag(message)
    correct += int(predicted == expected)
    print(f"predicted={predicted}, expected={expected}")

print(f"Correct decisions: {correct}/{len(examples)}")
```

Expected output:

```
predicted=True, expected=True
predicted=False, expected=True
predicted=True, expected=False
predicted=False, expected=False
Correct decisions: 2/4
```

The function checks a condition. The loop compares that result with the illustrative expected label. `int(True)` becomes 1 and `int(False)` becomes 0, so the counter increases only for a correct result. `len(examples)` is the number of examples. **Nothing in this program learns from the labels.** They are used for checking the rule, not for fitting parameters.

Try adding a second phrase to the rule. Predict which examples will change before running it. Then write at least two new examples, including a legitimate message quoting that phrase. This experiment makes a common trade-off visible: a broader rule may detect more unwanted messages while also creating false alarms.

The four cases deliberately expose weaknesses. They do not estimate real-world performance. To make such an estimate, you would need a much more representative evaluation set and a carefully specified labeling policy.

13. Practice: explain, decide, and investigate

Try answering before expanding a solution. The aim is to make your reasoning explicit, not to memorize a label.

Exercise 1: explain it without buzzwords

Explain AI to a colleague in three sentences. Include one task, one possible method, and one limitation.

▼ Show a sample answer

AI is a field concerned with building systems that perform tasks such as recognizing information, using language, or choosing actions. For example, a support tool can use a model fitted on labeled messages to suggest a category for a new request. Its suggestion still needs evaluation because unfamiliar wording or misleading patterns can produce mistakes.

Other answers are valid. Check that you have not equated all AI with chatbots or claimed that every AI system learns.

Exercise 2: choose the first approach

For each task, choose an initial approach and explain why: add 25 transaction amounts; detect an absent customer ID; group varied complaint descriptions; draft a handover note from confirmed events.

▼ Show the reasoning

Use arithmetic code or a spreadsheet for the sum. Use a validation rule for the absent ID. For complaints, first clarify whether the groups are known categories: a classifier suits predefined labels, while exploratory grouping is a different task. Compare a simple baseline before adding a learned approach. For the handover note, start with a template or a generative draft and check every factual statement against the supplied events.

Exercise 3: calculate the mistakes

Using the eight-message table, choose a threshold of **0.60**. Which messages are flagged? How many false positives, false negatives, and correct decisions occur?

▼ Show the calculation

A, B, C, and D are flagged. A, B, and D are correct spam detections. C is one false positive. F is one false negative. E, G, and H are correctly accepted. There are six correct decisions out of eight, so accuracy is 75%. That still leaves two different kinds of mistake to consider.

Exercise 4: identify unsupported claims

A draft says: "The authentication service failed at 10:02 because its connection pool was exhausted. Restarting it permanently fixed the issue at 10:07." Compare this with the source note in Section 8.

▼ Show the evidence check

10:02 is the first recorded user report, not a proven failure start. Connection-pool exhaustion is not in the note. The restart happened at 10:07, but the successful login check was at 10:12. Permanent resolution is unproven, and the cause remains under investigation. Rewrite the statement using the observed events and preserve the uncertainty.

Exercise 5: distinguish training from inference

A developer fits a ticket classifier using 2,000 labeled messages. Tomorrow it categorizes 50 new messages. A user corrects one category. Identify training, inference, and what remains unknown.

▼ Show the answer

Fitting the model using the 2,000 labeled messages is training. Categorizing the 50 new messages is inference. The correction is feedback. We cannot conclude that it changed the model: we need to know whether the application stores it, updates rules, retrains later, or supports some form of online learning.

Exercise 6: a score without useful behavior

Imagine 100 reviewed messages contain only two spam messages. A system calls every message legitimate. How accurate is it, and why is that not enough?

▼ Show the calculation

It gets 98 out of 100 labels correct, so accuracy is 98%. It detects neither spam message. The example shows how accuracy can hide failure on an infrequent class. A useful evaluation must examine spam detection and false alarms separately, alongside the actual application requirements.

Exercise 7: make a miniature system design

Choose one task from your work or daily life. Write down the input, desired output, a simple baseline, one possible AI component, two likely failure cases, and how a person would check the result.

▼ Show a worked example

Task: draft a release handover. Inputs: approved change notes and deployment steps. Output: a structured draft preserving versions, dates, and unresolved issues. Baseline: a fixed template filled manually. AI component: a language model that organizes supplied notes. Failures: inventing a completed verification or copying the wrong version. Check: compare each material claim with its source and validate version identifiers using code. Keep release approval outside the drafting component.

Exercise 8: explain the experiment

When you lower the threshold in the spam experiment, did the system learn? Explain what changed and what stayed the same.

▼ Show the answer

No learning occurred. The eight scores and labels remained fixed. The decision rule used a different cutoff, changing which scores were flagged. In a real application, tuning this cutoff is part of designing the decision policy; it does not by itself refit the model's parameters.

14. Check your understanding

Answer these short questions, then inspect the explanations. A correct selection is useful, but being able to explain why the alternatives are wrong is a stronger checkpoint.

01 Which statement describes AI most accurately?

- A. Every program that automates a task is AI.
- B. AI includes methods based on search, rules, and learning.
- C. AI and chatbots mean the same thing.

Answer: B. AI is a broad field. Chatbots are one application, and simple automation alone does not establish an AI method. Rules and search can be part of classical AI.

02 A model assigns a score of 0.82. What can we conclude?

- A. The event has exactly an 82% chance of occurring.
- B. The application must automatically act.
- C. We need the score definition and decision policy to interpret it.

Answer: C. Scores need a defined meaning. Probability calibration is a separate property, and an action is chosen by an application policy.

03 Which operation is training?

- A. Adjusting model parameters using examples and an objective.
- B. Supplying a new document in the input.
- C. Moving a decision threshold from 0.70 to 0.40.

Answer: A. Training fits parameters. New context changes the information supplied, while changing a cutoff changes the policy around an output.

04 A restart was followed by recovery. What should a factual report say?

- A. The restart proves the root cause.
- B. Record the sequence and keep the cause unknown unless supported.
- C. The fault is permanently fixed.

Answer: B. The sequence is evidence of what was observed. It does not establish the original cause or a permanent fix.

05 What is the best initial approach to adding known amounts?

- A. A large language model with no checking.
- B. A classifier trained on invoices.
- C. A deterministic calculation in code or a spreadsheet.

Answer: C. The arithmetic operation is precisely specified. Exact computation provides a clear, checkable baseline without needing a learned model.

15. Revision notes and glossary

AI describes a field of capabilities and methods. It includes approaches based on search, rules, planning, and learning. ML is an approach within AI, not a replacement name for the whole field.

Start with input, task, output, and success criteria. These make a problem concrete enough to design and evaluate.

A model is part of an application. Data handling, permissions, calculations, policies, review, and monitoring affect whether the complete system works.

Training fits parameters; inference uses a fitted model. Supplying new context, recording feedback, and changing a threshold are distinct from updating parameters through training.

A score is not an action or a guaranteed probability. A policy turns scores into decisions. Different errors can have different costs.

Generative output requires evidence checks. A fluent incident summary can invent a cause or misrepresent a timeline. Correct structure does not guarantee correct facts.

A simple baseline gives you a comparison. Use exact code for exact operations, and introduce AI where it improves a clearly defined task.

Evaluate on relevant new examples. Inspect mistakes and their consequences, not only an average score.

Mini glossary

Term	Plain-language meaning
AI	The field of building systems with capabilities such as perception, reasoning, planning, language, and learning
Machine learning	Fitting models using data to perform a task
Algorithm	A specified computational procedure
Model	A representation used to produce outputs; an ML model has state fitted using data
Parameter	An adjustable internal value of a model
Feature	An input property represented for a model
Label	A supplied target category or answer associated with an example
Dataset	A collection of examples used for development or evaluation
Training	The process of fitting a model

Term	Plain-language meaning
Inference	Running a fitted model on an input
Loss	A numerical measure of mismatch used in a training objective
Generalization	Useful performance on relevant new cases
Threshold	A cutoff that turns a score into a decision
False positive	A case incorrectly flagged as belonging to the positive class
False negative	A positive case that the system misses
Baseline	A reference approach used for comparison
Grounding	Tying an answer to appropriate supporting information
Hallucination	Incorrect or unsupported generated content presented as an answer
Token	A unit of text represented by a language model
Validation	Checking that data or output satisfies requirements; later chapters also use “validation set” for development evaluation

Before moving to Chapter 2

Without looking back, explain why the phrase rule is not learning, why lowering a threshold changes errors, why the incident draft cannot claim a root cause, and why a calculator may be the best tool for a total. If one explanation is difficult, revisit the relevant example and try again.

Your final task: Draw a small system for an everyday problem. Label the input, the AI component if there is one, the ordinary code, the human decision, and the evaluation method. You understand this chapter when you can explain the connections, not merely name the boxes.

16. Sources and where we go next

The examples, synthetic data, diagrams, exercises, and calculations in this chapter were created for this handbook. The following primary educational sources support the definitions and provide optional further reading. The chapter is an introductory synthesis, not a reproduction of any source.

Poole and Mackworth: [What is Artificial Intelligence?](#) - an agent-oriented treatment of AI and intelligent behavior.

Google: [What is Machine Learning?](#) - an introduction to fitting models and common ML tasks.

Google: [Accuracy, recall, precision, and related metrics](#) - further explanation of classification errors and measures.

scikit-learn: [Metrics and scoring](#) - a technical reference for later implementation work.

Next in the plan: Chapter 2 connects AI, ML, deep learning, NLP, generative AI, LLMs, RAG, agents, and AGI in one carefully explained map. It is planned, not yet written.